Problem 2

Feedforward neural networks and training basics

Backprop: what it computes and what it needs

Q1 (a) With intermediate values computed from a forward pass of a neural network, backpropagation computes the gradients of losses by differentiating with chain rule, such that those gradients are minimized in order to generate an optimized outcome of the neural network, as known as "gradient descent".

Q1 (b) Loss and the computed θ_t (the model's parameter) at state t should be cached from a forward pass. θ_t captures a model's parameters. In a purely linear layer, parameters are activated based on $y = wx + b$, where x is the data sample, y is its label, and the rest are parameters, i.e., represented by θ_t. A neural network is trained to find $\theta_t = \{w, b\}$ by maximizing $\hat{y} = wx + b$. Assume the values increase linearly. To achieve this, mathematically the maxima could be reached when $\frac{dy}{dx} = 0$, i.e., when the gradients stop increasing or changing. $\frac{dy}{dx} = w$ is decreasing and so, we know w is the weight / gradient in a linear layer. A forward pass calculates the change of parameter w. By comparing with the true labels in a training set, it also helps estimating how close the new label derived from w is to the true label. In a whole picture, it is also equivalent to $KL(P||Q) = -\sum_{i=1}^{N} p_i \log \frac{p_i}{q_i}$, where P represents the distribution in the true labels set while Q represents the approximated $\hat{y}$ from the activation function. The KL Divergence is the difference between the true dataset and the approximated dataset, and therefore, represented as the "loss". While the backpropagation evaluates an optimal $\hat{y}^*$ from gradient descent, we need the loss and the intermediate $\theta_t = \{w, b\}$ to be cached from the previous process, which is the forward pass, so that the neural network knows where it could converge from. ReLu, abbreviated as "rectified" linear unit, is an alternative to a purely linear unit which avoid vanishing gradient where $\frac{dy}{dx} < 0$, by finding $\max(0, \hat{y})$ where some $\hat{y}$ function might yield a negative value due to non-linearity which is meaningless to keep in the network's learning.

Mini-batch training and SGD as a noisy estimator

Q2 (a) Given empirical risk (or the loss) = $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} l_i(\theta)$; and, $\hat{g}_B(\theta) = \frac{1}{B}\sum_{i=1}^{B} \nabla l_{ib}(\theta)$, as the change of losses, and so, by definition, $\nabla J(\theta) = \hat{g}_B(\theta)$.

$$\Rightarrow \sum_{i=1}^{N} \nabla\, l_i(\theta) = \frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta)$$

An expectation is equivalent to the mean of all samples from a particular random variable.

$$\Rightarrow \sum_{i=1}^{N} \nabla\, l_i(\theta) = \mathrm{E}_B[\nabla l_{ib}(\theta)]$$

Meanwhile, recall that $\nabla J(\theta) = \sum_{i=1}^{N} \nabla\, l_i(\theta) = \hat{g}_B(\theta)$.

$$\Rightarrow \mathrm{E}[\hat{g}_B(\theta)] = \mathrm{E}_B[\nabla l_{ib}(\theta)] = \nabla J(\theta) \blacksquare$$

Q2 (b) $Var[\hat{g}_B(\theta)] = \mathrm{E}[\hat{g}_B(\theta)^2] - \mathrm{E}[\hat{g}_B(\theta)]^2$. By definition, the curvature (second derivative) of a random variable is the change of change of slope, which also represents its correlation or variance. Therefore, we also have: $Var[\hat{g}_B(\theta)] = \nabla^2 J(\theta)$.

Q2 (c) To see how the gradient changes with respect to B, the batch size, as mentioned in (b), we can derive from the curvature, where $Var[\hat{g}_B(\theta)] = \nabla^2 J(\theta)$.

$$\Rightarrow Var[\hat{g}_B(\theta)] = \mathrm{E}[\hat{g}_B(\theta)^2] + (\mathrm{E}[\hat{g}_B(\theta)])^2 = \nabla^2 J(\theta)$$

$$\Rightarrow \mathrm{E}\left[(\frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta))^2\right] - \nabla J(\theta)^2 = \nabla^2 J(\theta)$$

$$\Rightarrow \frac{(\frac{1}{B}\sum_{i=1}^{B} \nabla\, l_{ib}(\theta))^2}{N} - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

Meanwhile, in machine learning terms, the squared mean of losses from a batch over all data samples is equivalent to variance of losses per sample, i.e., $Var(\nabla\, l_i(\theta))$.

$$\Rightarrow Var\big(\nabla\, l_i(\theta)\big) - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

With the given assumption $Var\big(\nabla\, l_i(\theta)\big) = \sum \theta$, we have:

$$\sum \theta - \left(\sum_{i=1}^{N} \nabla\, l_i(\theta)\right)^2 = \sum_{i=1}^{N} \nabla^2 l_i(\theta)$$

From both sides of the equivalence, the curvature of loss, which is the co-variance of $\hat{g}_B(\theta)$, does not have any relationship with B. This could also be visualized in a mathematical manner of $\nabla f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \cdots \\ \frac{\partial f}{\partial \theta_n} \end{bmatrix}$ given any arbitrary function $f(\theta)$, with parameter set θ. The matrix only depends on the number of features in the model θ. Therefore, the gradient noise covariance changes irrespective of the batch size.

Training Curves: Underfitting vs. Overfitting

Q3 (a) Training loss decreases because the model starts converging and learning within the training data. Validation loss fluctuates because the model could not adapt the trend in the training set to the validation set. Thus, this is a phenomenon of **overfitting**, where the model learns too well from the training data (similar to "memorizing" samples) but lacks generalizability on unknown data.

Q3 (b) (i) In terms of data, one could observe from the empirical risk $J(\theta) = \frac{1}{N}\sum_{i=1}^{N} l_i(\theta)$, where N represents the number of samples $l_i(\theta)$ represents losses from each sample, under the model θ. The goal of gradient descent, in any machine learning model, is to reduce $\nabla J(\theta)$, i.e., to find θ when $\nabla J(\theta) = 0$. Mathematically, we then have $\sum_{i=1}^{N} \nabla\, l_i(\theta) = 0$. In this expression, the larger N is, the more loss terms are added to result in 0, and in turn, each $\nabla\, l_i(\theta)$ will become smaller and eventually reach 0. Applying in a machine learning setting, we could provide more samples such that the model has more room to converge better.

Q3 (b) (ii) In terms of the model, one could apply regularization techniques like **early stopping**. Overfitting generally means the training process takes unnecessarily long time. Early stopping interferes the training loop once the gradient of losses reaches or very similar to 0. This avoids the model over-generalizing on the training data while allowing it to just make it converged.

Q3 (b) (iii) In terms of optimization, one could apply a suitable optimizer like Adam or AdamW. From $\nabla J(\theta) = \sum_{i=1}^{N} \nabla\, l_i(\theta)$, one proven way in (a) to minimize it is to increase the number of samples. However, provided that N stays constant, another way is to apply another type of loss function $l_i(\theta)$. The current one is stochastic gradient descent, which updates θ parameters according to gradient of losses. Another type is **Adam** (Adaptive Moment Estimation), which is to compute individual adaptive learning rates for different θ parameters with the first and second moments of the gradients. A variant, **AdamW**, applies weight decay on it.

Two signs of data leakage or evaluation bugs

Q4) Data leakage typically refers to a misuse of data. For example, test samples which are supposed only for evaluation are being trained as weights of parameters in the model. While a subset of samples is both being trained and evaluated, there is a high risk where the model predicts it by memorizing its label during training. This is probably caused by a logic error in the training methodology. Memorizing leads to overfitting. Identifying the issue is as simple as identifying overfitting. After training, one could plot convergence graphs, based on the change of accuracies, losses, or KL divergences, on both the training set and a validation set. For example, if the loss appears relatively higher on the validation set than the training set, or convergence on the training set happened for many epochs, the model is overfitting the data. To solve this, I would first check how the dataset is split and how those splits are used during the training, validation, and evaluation (i.e., testing) pipelines. If those sets are ensured to be in the right place without any reuse, I would check whether any individual sample occurs in both splits, e.g., training and test set. To achieve

this, a simple INTERSECTION operation between sets, based on Set Theory, in case returning any data samples, would visualize the error.

Methodologically, gradient flowback could mistakenly happen only in particular layers. Dropout and Batch Normalization are commonly susceptible ones that could mistakenly switch between training mode, where parameters would be updated via backpropagation, and evaluation / prediction mode. This could affect the entire neural network's ability to correctly learn even one sample. It could firstly be observed from tools like TensorBoard, to see how weights and gradients flow. A simple experiment is to overfit a small subset of data. On for example a subset of only 5 training samples, I would launch a prediction with the model. If it fails to accurately predict those 5 samples, given a small number of samples, the training process is likely faulty.

Part B: Convolutional Networks

Fully connected vs. convolution

Q5 (a) From the fully connected layer mapping $\mathbb{R}^{CHW} \rightarrow \mathbb{R}^{M}$, with the convolution layer mapping $x \in \mathbb{R}^{CHW} \rightarrow y \in \mathbb{R}^{MHW}$, we can derive the full image's size is $H \times W$, where H is the height of the image and W is the width of the image. Meanwhile, the number of channels is C. Given the kernel size = $K \times K$, i.e., a square, the count of parameters for a kernel filter = $K \times K \times C$. This is a locally connected network which helps the entire neural network to learn the entire image's pixels. From the output layer, while $H \times W$ is the image's dimension, the number of filters required is M. With stride = 1, the convolutional kernels/filters slide 1 block away from its current position until the margins of the image. Given the number of filters required at the output layer, and the parameters required per filter, we can derive the total parameter count = $M \cdot K^2 \cdot C$.

Q5 (b) Inductive bias generally refers to a structure that is infused into the model. Such a structure (or an intuition) allows functions to learn from a constrained hypothesis space. With much inductive bias, a model relies more on the prior belief rather than purely dependent on data samples. A kernel (or filter), in a convolutional network, is a specific data type, usually a grid whose length and width are divisible by the full image's ones, that slides through the entire image, to learn pixels all over the image stepwise, within a locally connected network. During one slide, those layers are locally connected to learn a specific region of pixels, while weights are shared amongst all slides or transitions. This in turn allows the neural network to efficiently learn the whole image, which might be highly dimensional, in a more computationally efficient way. Because of shared weights across filters, the number of parameters needed for a convolutional layer / network, is irrespective to the image's dimensions $H \times W$.

Q5 (c) Convolutional inductive bias might not be appropriate in action recognition tasks in **video** where temporal dimension is involved but shift-invariant is not expected in time, such as distinguishing between "picking a cup up" and "putting a cup down". That is because the order of events holds significant information for the data.

Locality/translation and pooling invariances

Q6 (a) Translation equivalence in a model suggests that, in a model $f(x)$, applying translated features into the model has the same effect as translating the output of original features through the network, which means: $f\big(translate(x)\big) = translate(f(x))$. A CNN uses local context windows that allows the model to learn objects (represented in the image) in the real world, with noises such as different orientations for edge detection. To account for those noises, such as an object's positional invariance, a CNN develops translation equivalence such that it accurately classifies an object regardless of such an invariance.

Locality suggests that a CNN forms a locally connected network with a kernel. Those kernels use shared weights, making the parameter count irrespective to an image's dimension and thus, enhancing the computational efficiency with fewer parameters possible.

Q6 (b) Pooling evaluates the outcome with either a maximum feature value: $y_j = \max_{j \in N(j)} h_j$ (max-pooling); or with the mean of feature values: $y_j = \frac{1}{|N|}\sum_{j \in N(j)} h_j$ (mean-pooling). Those evaluations provide only a summary of a vector of feature values, regardless of exact position, rotation, as well as where the edge is. This provides stability for a model to learn an object regardless of those noises (invariances). Pooling could also happen across channels, orientations, or edges. In such a way, pooling introduces new invariances as new forms or arrangements of feature values are produced. It does not guarantee stable learning but the ability to learn different forms of the same object.

Q6 (c) As suggested in (a), translation equivalence means $f\big(translate(x)\big) = translate(f(x))$ for any model $f(x)$. Stride = 2 means a kernel filter slides 2 blocks away from its previous location during convolutional transition. If a newly transformed feature vector not translate-equivalent, the kernel should fail to slide through every pixel in the image. This could happen when pooling condenses the feature vector in a way that its length is not divisible by the kernel's length. Down-sampling could be an example of changing the size of the feature vector where you only keep a subset of feature values.

Residual connections: why do they help?

Q7) Residual connection enables the model to optimize by learning its own architecture implicitly. In traditional encoder-decoder architecture, an original image passes through multiple layers, possibly down-sampled, and produces a features vector in a very low dimension, via an encoder; followed by a decoder which tries to produce the original vector from the downsized one. A U-Net typically incorporates a "skip connection", which defines whichever layer should be skipped. Those outputs that are passed through the skip connection are defined as a "residual". From a residual block $y = x + F(x;\theta)$, when the gradient is too small after a specific layer, a skip connection effectively avoids such vanishing gradients in deep networks as it provides a direct path of the information (i.e., gradient) flow that skips that layer. Practically, ResNet is an example

where it often uses a skip connection to avoid passing through layers that do not contribute meaningfully to a specific task. By classifying very similar images, encoders often produce very similar outcomes. A skip connection could be useful to determine which layers in the model are to be skipped, so as to produce distinguishable and meaningful encoded vectors.

Part C: Recurrent networks (RNNs) and sequence modeling

RNN Markov structure and what it means

Q8 (a) As a "Markov" encapsulates a time state during a transition, conditional independence suggests that a hidden layer in a recurrent neural network (RNN) computes the value at the t^{th} hidden state by only referring to the most recent $(t-1)^{\text{th}}$ hidden state's value. It efficiently reduces memory size of storing long dependences.

Q8 (b) Even though a forward pass only refers to 1 previously seen state in a hidden layer, the RNN backpropagates from the output state t back to the very first state, algebraically means $\frac{\partial x_{out}[t]}{\partial x_{in}[0]}$, which updates the weights of those hidden layers in the network to fit the samples well. By substituting the latest h_{t-1} in every h_t state, the weight term W_h, which is a constant, multiplies by itself, making the order of Φ equivalent to the current state t. Like in the state $t = 1$, $h_1 = \Phi(W_h h_0 + \mathrm{U}x_1 + \mathrm{b})$, followed by state $t = 2$, where $h_2 = \Phi(W_h h_1 + \mathrm{U}x_2 + \mathrm{b}) = \Phi(W_h \Phi(W_h h_0 + \mathrm{U}x_1 + \mathrm{b}), + \mathrm{U}x_2 + \mathrm{b})$. The order of W_h becomes 2, i.e., ${W_h}^2$, and so on. Meanwhile, Φ is also called twice in this state, meaning that it also corresponds to the order of W_h. By doing gradient descent, the backpropagation tries to minimize the sum of individual losses in each state, differentiate the expression in h_t, and thus, helps a RNN represent and learn a long range of states back in time.

Simple RNN forward and backward

Q9 (a) When $t = 1$, hidden state $h_1 = \tanh(W_{xh}x_1 + W_{hh}h_0 + b_h) = \tanh(W_{xh}x_1 + b_h)$, and $\hat{y} = W_{hy}h_1 = W_{hy} \cdot \tanh(W_{xh}x_1 + b_h)$.

When $t = 2$, hidden state $h_2 = \tanh(W_{xh}x_2 + W_{hh}h_1 + b_h) = \tanh(W_{xh}x_2 + W_{hh} \cdot \tanh(W_{xh}x_1 + b_h) + b_h)$, and $\hat{y} = W_{hy}h_2 = W_{hy} \cdot \tanh(W_{xh}x_2 + W_{hh} \cdot \tanh(W_{xh}x_1 + b_h) + b_h)$.

As derived from 8(b), the order of weight term increases as the current state, as well as the order of the hidden layer's function, in this case, the hyperbolic tangent. From state 1 to T, therefore, a forward pass could derive: $h_T = \tanh(W_{xh}x_T + \tanh^{T-1}(W_{xh}x_{T-1} + b_h) + b_h)$ and $\hat{y}_T = W_{hy}h_T = W_{hy} \cdot \tanh^T(W_{xh}x_T + b_h)$.

Q9 (b) From 9 (a), we have:

$$h_{t+1} = \tanh(W_{xh}x_{t+1} + W_{hh}h_t + b_h) = \tanh(W_{xh}x_{t+1} + \tanh^t(W_{xh}x_t + b_h) + b_h)$$

$$\hat{y}_{t+1} = W_{hy}h_{t+1} = W_{hy} \cdot \tanh^{t+1}(W_{xh}x_{t+1} + b_h)$$

From the original expressions $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$ and $\hat{y}_t = W_{hy}h_t$, we could derive the loss: $L(h_t) = \sum_{t=1}^{T} l(\hat{y}_t, y_t) =$

$\frac{\partial \hat{y}}{\partial x} = \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial x} = W_{hy} \cdot \operatorname{sech}^2(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \cdot W_{xh}$. To find the change of losses, we have: $L'(h_t) = \frac{\partial L}{\partial h_t} = -2W_{hy} \cdot \operatorname{sech}^2(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \cdot \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \cdot {W_{xh}}^2$.

Therefore, $\frac{\partial L}{\partial h_{t+1}} = L'(h_{t+1}) = \frac{\partial L}{\partial h_t}|_{t=t+1} = -2W_{hy} \cdot \operatorname{sech}^2(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \cdot \tanh(W_{xh}x_{t+1} + W_{hh}h_t + b_h) \cdot {W_{xh}}^2$ ■

Note that sech is a bell-shaped curve similar to a normal probability density function's curve and so do sech^2.

Q9 (c) In 9 (a), it is derived that h_T and $\hat{y}_T$ will have nested tanh function calls where the number of times (i.e., the order of tanh) depends on T, for $t = 1, \dots, T$ states. Gradient $\frac{\partial \hat{y}}{\partial x}$, which is also the loss, shrinks with a higher order of tanh, and therefore, gradient vanishes and tends to 0. The W_{hh} term will exist in each tanh function call, so the magnitude of W_{hh} strictly depend on the number of states T. If it is smaller than 0, $\tanh^{-1}$ happens to be a reciprocal, so W_{hh} will be inversely proportional to h_t and thus, the loss $L(h_t)$. So, it will grow when there are less states T. This will lead to exploding gradients, which tends to infinity. On the contrary, W_{hh} is scaling proportionally with h_t and $L(h_t)$ with a positive power of tanh. So, in this case, it will grow only with more states T. If it is between 0 and 1, tanh happens to be a root, but still, scales proportionally with T. Either exploding or vanishing gradients makes the RNN struggle to learn long-term dependencies.

If the weight (i.e., the bias term) increases with more observed states, there will likely be a memory issue, where the model tries to remember everything, it sees. To improve, typically a LSTM (Long-Short Term Memory) layer could be used. It helps the neural network to learn whatever states to remember or to forget.